The Effect of Feature Scaling on Coefficients in Machine Learning Models
Daniel South
2026-05-20

Feature Scaling is in important technique in Machine Learning, but there's a caveat that may cause confusion unless it's clearly understood. Feature scaling scales not just the data, but also the coefficients calculated by the model. Before those calculations can be used for practical purposes, the need to be rescaled to the original dimensions of the data set.

-

Feature scaling maps all numeric features in a data set to a similar range to ensure that each feature is given the similar consideration by the model.

A data set on home sale prices might contain large value (square feet), small values (number of bathrooms), and other values that fall between these extremes (age of the structure).

Mapping all values to a similar range using min/max scaling (0-1) or standard scaling (mean=0, variance=1) ensures that the larger numbers won't dominate the training of the model.

This demonstration uses a simple, one-feature model to illustrate the impact of feature scaling on model coefficients and how to rescale the coefficients back to the origional scale of their respective features.

In [1]:
```
# What is the impact of scaling on Linear Regression models?
# How can we use the parameters of a model fitted to scaled data when making predictions with new data?
# Can we use the slope(s) and intercepts without any adjustments?

# To demonstrate what can go wrong when fitting a model to scaled data, we'll
# use a very simple data set with a single feature (square footage) to predict home prices.

# The data set is fake, but it's simple enough to clearly demonstrate the impact of scaling
# on model fitting in an easy to understand scenario.
```

In [2]:
```
import numpy as np
import pandas as pd

import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

from sklearn.preprocessing import StandardScaler

RANDOM_STATE=20
```

In [3]:
```
data_file = "home_prices_by_square_foot_simulated.csv"
```

In [4]:
```
df = pd.read_csv(data_file)
df.head()
```

Out[4]:

	Price	Square Feet
0	200000	2000
1	205000	2075
2	210000	2050
3	215000	2130
4	220000	2200

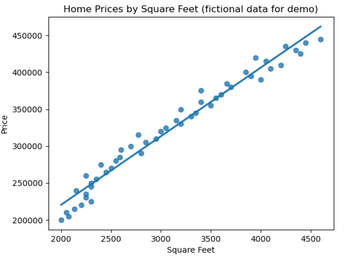
In [5]:
```
# Split the X and y values into different data frames
```

In [6]:
```
y = df['Price']
X = df.drop('Price', axis=1)
print(y.head())
print(X.head())
#print(df.head())
```

```
0    200000
1    205000
2    210000
3    215000
4    220000
Name: Price, dtype: int64
   Square Feet
0         2000
1         2075
2         2050
3         2130
4         2200
```

In [7]:
```
# Scatter plot the data to see if it's roughly linear

sns.regplot(x=X, y=y, ci=None)
plt.title("Home Prices by Square Feet (fictional data for demo)")
plt.show()
```



In [8]:
```
# Split the data randomly into train and tests sets (80 percent for training, 20 percent for test)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=RANDOM_STATE, train_size=0.8)
```

In [9]:
```
print(len(X_test))
```

```
10
```

In [10]:
```
# Fit a linear regression model to the data WITHOUT scaling

model1 = LinearRegression()
model1.fit(X_train, y_train)

y_predicted = model1.predict(X_test)

mse = mean_squared_error(y_test, y_predicted)
rmse = np.sqrt( mse ).round(2)

print(f"The Root Mean Squared Error is {rmse}")
```

```
The Root Mean Squared Error is 14668.97
```

In [11]:
```
# The root mean squared error is under $15,000.00
# We can interpret this as a reasonably good fit for the data set.
```

In [12]:
```
# Get the intercept for the line and the coefficient for the square foot feature.

intercept = model1.intercept_
slope = model1.coef_[0]

print(f"The intercept is {round(intercept, 2)}")
print(f"The coefficient for the square foot parameter is {round(slope, 2)}")
```

```
The intercept is 33542.99
The coefficient for the square foot parameter is 93.19
```

In [13]:
```
print("Extimate home prices based on square feet using the intercept and slope from the model.")

house_size_list = [3000, 6000, 8000]
for sqfeet in house_size_list:
    price = intercept + (slope * sqfeet)
    print(f"For a house with {sqfeet} square feet, the estimated price is ${round(price)}")
```

```
Extimate home prices based on square feet using the intercept and slope from the model.
For a house with 3000 square feet, the estimated price is $313119
For a house with 6000 square feet, the estimated price is $592695
For a house with 8000 square feet, the estimated price is $779079
```

In [14]:
```
#
# Let's create a second model with scaled data and compare the coefficients with those of the first model.
#
# Scale the data and fit the feature data to the scaler.
# This standardizes the data to have a MEAN of 0 and a VARIANCE of 1
#

std_scaler = StandardScaler()

# Fit the standard scaler to the training data only in order to prevent data leakage
# i.e. so the model is not influenced by test data.

std_scaler.fit(X_train)

# Scale both the training features and the test features using the fitted scaler.
# Note: The Response (y) is typically not scaled
X_train_scaled = std_scaler.transform(X_train)
X_test_scaled = std_scaler.transform(X_test)
```

In [15]:
```
# Create a new linear regression model and fit it using the scaled data.

model2 = LinearRegression()
model2.fit(X_train_scaled, y_train)

y_predicted2 = model2.predict(X_test_scaled)

mse2 = mean_squared_error(y_test, y_predicted2)
rmse2 = np.sqrt( mse2 ).round(2)

print(f"The Root Mean Squared Error is {rmse2}")
```

```
The Root Mean Squared Error is 14668.97
```

In [16]:
```
# Good news! The RMSE is the same for the scaled model than for the model trained with unscaled data.
# How about the slope and intercept?
```

In [17]:
```
intercept2 = model2.intercept_
slope2     = model2.coef_[0]

print(f"The intercept is {round(intercept2, 2)}")
print(f"The coefficient for the square foot parameter is {round(slope2, 2)}")
```

```
The intercept is 327750.0
The coefficient for the square foot parameter is 69402.42
```

In [18]:
```
# The slope and intercept from the scaled model are massively different.
# If we were to attempt to predict housing prices using these values, the results would be ridiculously inaccutate.

print("Calculating home prices using the slope and intercept from the scaled model leads to ridiculous results.")

house_size_list = [3000, 6000, 8000]
for sqfeet in house_size_list:
    price = intercept2 + (slope2 * sqfeet)
    print(f"For a house with {sqfeet} square feet, the estimated price is ${round(price)}")
```

```
Calculating home prices using the slope and intercept from the scaled model leads to ridiculous results.
For a house with 3000 square feet, the estimated price is $208535915
For a house with 6000 square feet, the estimated price is $416742281
For a house with 8000 square feet, the estimated price is $555547124
```

In [19]:
```
# Wow! That's a lot of money for a house!
# Something clearly has gone wrong. How do we fix it?
# How can we get coefficients that give us realistic predictions?
```

In [20]:
```
# In order to rescale the slope and intercept back to the scale of the original data set,
# we need to do some calculations courtesy of the brilliant minds at Scikit Learn.

slope_orig_scale = slope2 / std_scaler.scale_
print(f"The slope in its original scale would be {slope_orig_scale}")

intercept_orig_scale = intercept2 - np.sum( (slope2 * std_scaler.mean_) / std_scaler.scale_)
print(f"The intercept in the data's original scale would be {intercept_orig_scale}")
```

```
The slope in its original scale would be [93.19195624]
The intercept in the data's original scale would be 33542.99416172644
```

In [21]:
```
# Now the slope and intercept those of the unscaled data.

print("Extimate home prices based on square feet using the RESCALED intercept and slope from the SCALED model.")

house_size_list = [3000, 6000, 8000]
for sqfeet in house_size_list:
    price = intercept_orig_scale + (slope_orig_scale * sqfeet)
    price = price[0]
    print(f"For a house with {sqfeet} square feet, the estimated price is ${round(price)}")
```

```
Extimate home prices based on square feet using the RESCALED intercept and slope from the SCALED model.
For a house with 3000 square feet, the estimated price is $313119
For a house with 6000 square feet, the estimated price is $592695
For a house with 8000 square feet, the estimated price is $779079
```

In [22]:
```
# Mystery solved!
```

In [ ]:
```

```